

# 19장 함수 포인터와 VOID 포인터

## 차례

1. 함수 포인터와 void 포인터
2. main 함수로의 인자 전달

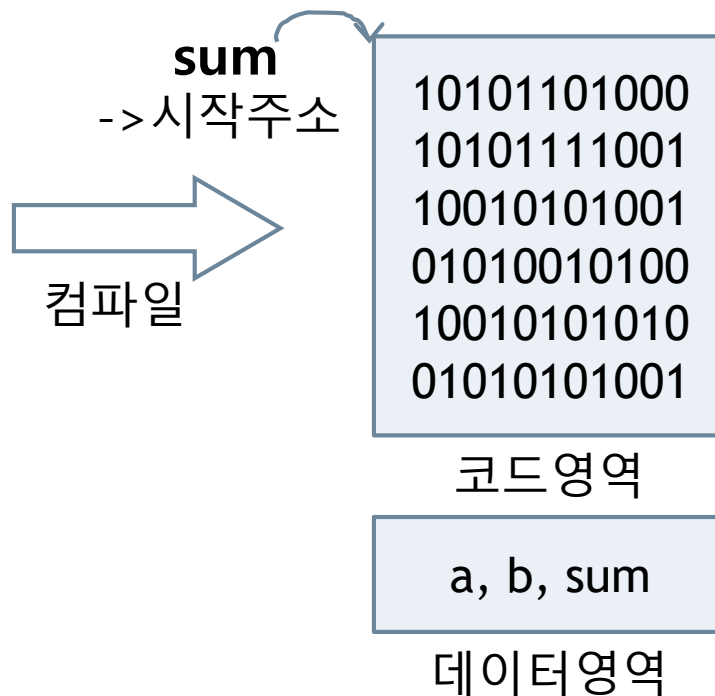


# 함수명의 의미

3

- 함수는 컴퓨터가 실행할 문장들의 모임 -> 기계어(이진코드)로 번역되어 메모리(코드영역)에 저장됨 -> 함수가 저장된 메모리의 시작 주소가 존재
- 변수에는 데이터가 저장되고 함수에는 실행코드가 저장
- **함수명 : 함수가 저장된 메모리 공간의 시작주소(상수)**

```
int sum(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```



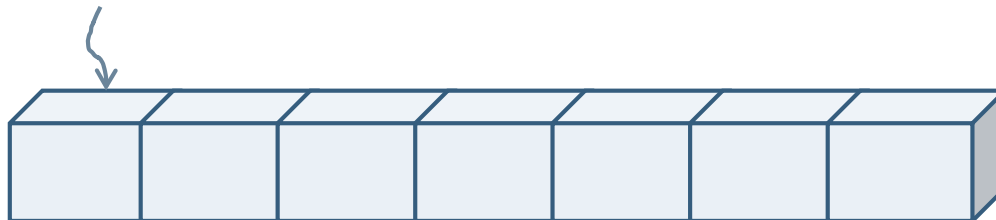


# 배열명과 함수명

4

- 배열명 -> 배열의 첫번째 원소의 주소 -> 배열의 시작주소
- 함수명 -> 함수의 시작주소

`int a[9];`    **a**->시작주소



```
int sum(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```

**sum**  
->시작주소

```
10101101000
10101111001
10010101001
01010010100
10010101010
01010101001
```



# 예제 1 : 함수명의 의미

5

```
#include <stdio.h>
int sum(int a, int b);
int main(void)
{
    printf("%d\n", sum(10,20)); //함수의 리턴값 출력
    printf("%p\n", sum);       //함수명 출력
    return 0;
}
int sum(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```

30

00007FF6561E13D9

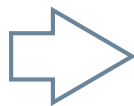


# 함수의 주소(함수명)의 자료형

6

- 함수의 자료형 -> 반환형과 매개변수의 자료형으로 정의
  - ▣ 반환형(매개변수자료형1, 매개변수자료형2, ...)
- 함수명의 자료형
  - ▣ (함수의자료형)\* -> (반환형(매개변수자료형1, 매개변수자료형2, ...))\*  
-> 반환형 (\*) (매개변수자료형1, 매개변수자료형2, ...)

```
int sum(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```



함수 sum의 자료형  
-> int(int, int)

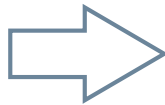
sum의 자료형  
-> (int(int, int))\*  
-> int (\*) (int, int)



# 함수명(함수의 주소)의 자료형

7

```
double abs(double a)
{
    if(a<0) a =-a;
    return a;
}
```



함수 abs의 자료형  
-> double(double)

abs의 자료형  
-> (double(double))\*  
-> **double (\*) (double)**

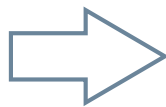


# 함수명(함수의 주소)의 자료형

8

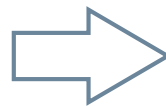
- 함수의 정의가 다르더라도 반환형, 매개변수의 자료형과 개수가 같으면 함수의 자료형은 같음

```
int sum(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```



sum의 자료형  
-> int (\*)(int, int)

```
int sub(int a, int b)
{
    int sub;
    sub = a - b;
    return sub;
}
```



sub의 자료형  
-> int (\*)(int, int)



# 함수 포인터

9

- 함수 포인터 : 함수의 주소(함수명)를 저장하는 변수

함수자료형\* 변수명;

-> 반환형 (\*)(매개변수자료형1, 매개변수자료형2, ...) 변수명;

-> 반환형 (\*변수명)(매개변수자료형1, 매개변수자료형2, ...);

```
int sum(int a, int b);
```

// 함수 선언

```
int (*fp1)(int, int);
```

// 함수 포인터 선언

```
fp1 = sum;
```

```
double abs(double a);
```

// 함수 선언

```
double (*fp2)(double) = abs;
```

// 함수 포인터 선언



# 함수 포인터

10

- 괄호가 없으면 반환형이 포인터인 함수의 선언임

```
int (*fp)(int, int); // 함수포인터 fp 선언  
int *fp(int, int);  // 반환형 int*, 매개변수 int, int인 함수 fp 선언  
int* fp(int, int);  // 반환형 int*, 매개변수 int, int인 함수 fp 선언
```

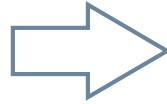


# 함수 포인터를 이용한 함수 호출

11

- 함수명 대신에 함수 포인터를 이용하여 호출가능

```
int sum(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```



```
int (*fp)(int, int) = sum;
int res;
res = sum(10,20);
res = fp(10,20);
```

- 포인터를 배열명처럼 사용하는 것과 유사

```
int arr[3] = { 15, 25, 35 };
int* ptr = arr;
printf("%d %d %d\n", ptr[0], ptr[1], ptr[2]);
```



## 예제 2: 함수 포인터를 이용한 호출

12

```
#include <stdio.h>
void SimpleAdder(int n1, int n2);
void ShowString(char* str);
int main(void)
{
    char* str = "Function Pointer";
    int num1 = 10, num2 = 20;
    void (*fptr1)(int, int) = SimpleAdder;
    void (*fptr2)(char*) = ShowString;

    SimpleAdder(num1, num2);    // 함수 호출
    ShowString(str);

    fptr1(num1, num2);          // 함수 포인터에 의한 호출
    fptr2(str);
    return 0;
}
```

10 + 20 = 30  
Function Pointer  
10 + 20 = 30  
Function Pointer



## 예제 2: 함수 포인터를 이용한 호출

13

```
void SimpleAdder(int n1, int n2)
{
    printf("%d + %d = %d \n", n1, n2, n1 + n2);
}

void ShowString(char* str)
{
    printf("%s \n", str);
}
```

10 + 20 = 30  
Function Pointer  
10 + 20 = 30  
Function Pointer



# 함수 포인터의 응용

14

- 함수를 인자로 받고 싶을 때 주로 함수의 매개변수에 활용함
- 주소에 의한 호출을 사용 -> 호출시 함수주소(함수명)만 받아서 실행
- 함수 내부에서 외부에서 정의된 함수를 마음대로 호출가능 하지만 함수 내부에서 외부에서 정의된 여러 개의 함수 중에 사용자가 선택 하여 실행하게 하고 싶을 때
- 매개변수를 함수 포인터로 선언하여 함수명 받아 실행 -> 코드사이즈 줄이고 유지 보수 용이



# 함수 포인터의 응용

15

- 함수는 어디서나 호출가능, 호출전에 선언해야 함

```
int add(int a, int b);
int sub(int a, int b);
int main()
{
    ...
    f1(); //함수 호출
    f2(); //함수 호출
    ...
}
// 함수 직접호출
void f1(void)
{
    res = add(10,20);
    ... // 동일 코드
}
void f2(void)
{
    res = sub(10,20);
    ... // 동일 코드
}
```

```
int add(int a, int b);
int sub(int a, int b);
int main()
{
    ...
    f(add); //함수명을 인자로 전달
    f(sub); //함수명을 인자로 전달
    ...
}
// 함수주소를 받아서 호출
void f(int (*fp)(int, int))
{
    res = fp(10,20);
    ... // 동일 코드
}
```



# 함수 포인터의 응용

16

- 함수에서 배열을 받아서 처리할 때 원리와 비슷

```
int sum(int a, int b);
int main()
{
    ...
    f(sum); //함수명을 인자로 전달
    ...
}
//주소에 의한 호출
void f(int (*fp)(int, int))
{
    res = fp(10,20);
    ...
}
```

```
int main()
{
    int a[10000];
    ...
    f(a); //배열명을 인자로 전달
    ...
}
void f(int* pt)//주소에 의한 호출
{
    res = pt[0];
    ...
}
```



## 예제 2: 함수 포인터 응용

17

- 함수의 일부기능을 외부에서 만들어 놓은 다른 함수들 중에 선택하여 구현하고 싶을 때

```
#include <stdio.h>
int big(int a, int b);
int small(int a, int b);
int get_value(int* a, int (*fp)(int, int));
int main(void)
{
    int arr[5] = {-1, 2, 10, 5, 7};
    int res;
    res = get_value(arr, big); //함수명을 인자로 전달
    printf("결과값은 : %d\n", res);

    Res = get_value(arr, small); //함수명을 인자로 전달
    printf("결과값은 : %d\n", res);
    return 0;
}
```

결과값은 : 10  
결과값은 : -2



## 예제 2: 함수 포인터 응용

18

```
int get_value(int* a, int (*fp)(int, int))
{
    int res = a[0];
    for (int i = 1; i < 5; i++)
        if (fp(res, a[i])) res = a[i]; // 함수포인터로 함수호출
    return res;
}

int big(int a, int b)
{
    if (a > b) return 1;
    else return 0;
}

int small(int a, int b)
{
    if (a > b) return 0;
    else return 1;
}
```

결과값은 : 10  
결과값은 : -2



## 예제 3: qsort 함수 활용

19

- 라이브러리함수에서 사용자 정의 함수를 요구하는 경우

```
#include <stdio.h>
#include <stdlib.h>
int compare1(const int* a, const int* b);
int compare2(const int* a, const int* b);
int main() {
    int data[10] = { 21, 10, 41, 33, 25, 17, 68, 27, 14, 29 };

    qsort(data, 10, sizeof(int), compare1); //오름차순
    for (int i = 0; i < 10; i++) printf("%d ", data[i]);
    printf("\n");

    qsort(data, 10, sizeof(int), compare2); //내림차순
    for (int i = 0; i < 10; i++) printf("%d ", data[i]);
    return 0;
}
```

```
10 14 17 21 25 27 29 33 41 68
68 41 33 29 27 25 21 17 14 10
```



## 예제 3: qsort 함수 활용

20

- 라이브러리함수에서 함수주소를 요구하는 경우

```
int compare1(const int* a, const int* b)
{
    return (*a - *b); //오름차순
}
int compare2(const int* a, const int* b)
{
    return (*b - *a); //내림차순
}
```

```
10 14 17 21 25 27 29 33 41 68
68 41 33 29 27 25 21 17 14 10
```



## 예제 4: 함수 포인터 응용

21

```
#include <stdio.h>
int WholsFirst(int age1, int age2, int (*cmp)(int n1, int n2));
int OlderFirst(int age1, int age2);
int YoungerFirst(int age1, int age2);
int main(void)
{
    int age1 = 20, age2 = 30;
    int first;

    printf("입장순서 1 \n");
    first = WholsFirst(age1, age2, OlderFirst);
    printf("%d세와 %d세 중 %d세가 먼저 입장! \n\n", age1, age2, first);

    printf("입장순서 2 \n");
    first = WholsFirst(age1, age2, YoungerFirst);
    printf("%d세와 %d세 중 %d세가 먼저 입장! \n\n", age1, age2, first);
    return 0;
}
```



## 예제 4: 함수 포인터 응용

22

```
int OlderFirst(int age1, int age2)
{
    if (age1 > age2) return age1;
    else if (age1 < age2) return age2;
    else return 0;
}
```

```
int YoungerFirst(int age1, int age2)
{
    if (age1 < age2) return age1;
    else if (age1 > age2) return age2;
    else return 0;
}
```

```
int WholsFirst(int age1, int age2, int (*cmp)(int n1, int n2))
{
    return cmp(age1, age2);
}
```

입장순서 1  
20세와 30세 중 30세가 먼저 입장!

입장순서 2  
20세와 30세 중 20세가 먼저 입장!



# void 포인터 필요성

23

- 포인터는 가리키는 변수의 자료형이 항상 정해져 있음 -> 가리키는 자료형 별로 포인터 변수가 필요 -> 포인터 변수가 많아짐

```
int a=10;           // int형 변수
double b=3.14;      // double형 변수
```

```
int* pi = &a;       // 정상
double* pd = &b;    // 정상
```

```
pd = &a;            // 오류
pi = &b;            // 오류
```



# void 포인터

24

- void포인터 : 모든 종류의 주소를 저장할 수 있는 변수
- void는 가리키는 자료형이 정해져 있지 않다는 의미 -> 모든 종류의 주소를 저장할 수 있다는 의미, 함수의 주소도 저장가능
- void 포인터 선언방법

```
void* 변수명;
```

```
// void포인터 선언 예
```

```
void* vp;
```



# void 포인터

25

- void 포인터는 하나의 포인터로 모든 종류의 주소를 저장할 수 있음

```
char ch='a';           // char형 변수
int in=10;             // int형 변수
double db=3.14;        // double형 변수
void SoSimpleFunc(void); // 함수선언
void* vp;              // void포인터변수

vp = &ch;              // char형 변수의 주소를 저장
vp = &in;              // int형 변수의 주소를 저장
vp = &db;              // double형 변수의 주소도 저장
vp = SoSimpleFunc;     // 함수의 주소도 저장
```

 예제 3

26

```
#include <stdio.h>
void SoSimpleFunc(void);
int main(void)
{
    int num = 20;
    void* ptr;

    ptr = &num;    // 변수 num의 주소 값 저장
    printf("%p \n", ptr);

    ptr = SoSimpleFunc;    // 함수 SoSimpleFunc의 주소 값 저장
    printf("%p \n", ptr);
    return 0;
}
void SoSimpleFunc(void)
{
    printf("I'm so simple");
}
```

```
000000FDC8D4F7D4
00007FF7206913CF
```



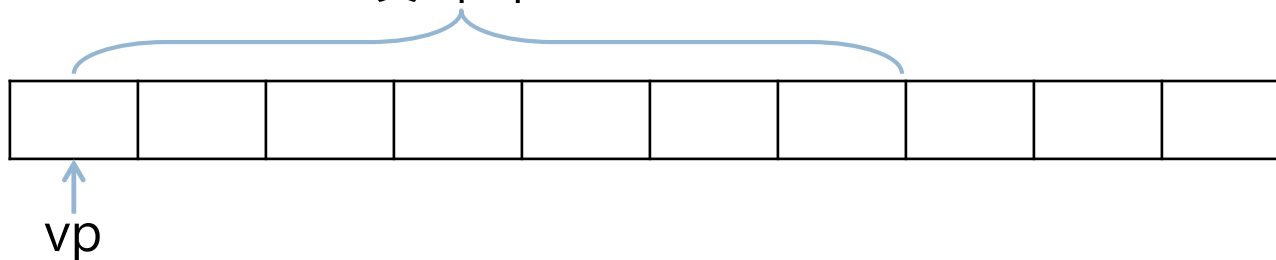
# void 포인터의 가접참조연산\*

27

- \*vp: 가리키는 자료형이 없으므로 포인터에 저장된 주소로부터 몇 바이트를 읽어야 하는지 알 수 없음

```
int in=10;           // int형 변수
void* vp = &in;      // void포인터변수
printf("%dWn", *vp); // 오류
vp++;               // 오류
*vp = 20;           // 오류
```

몇바이트?





# void 포인터의 간접참조연산\*

28

- void포인터에 연산을 적용할 때에는 반드시 강제 형변환하여 가리키는 자료형을 정의한 후 사용해야 함

```
int in=10;           // int형 변수
void *vp;            // void포인터변수
vp = &in;            // int형 변수의 포인터를 저장

printf("%d\n", *(int*)vp); // 형변환 후에 간접참조연산 가능
vp = (int*)vp+1;      // 형변환 후에 정수덧셈가능
*(int*)vp = 20;
```

 예제 4

29

```
#include <stdio.h>
int main()
{
    int a = 10;
    double b = 3.5;
    void* vp;

    vp = &a;
    printf("a: %d\n", *(int*)vp); // 강제형변환후 *연산

    vp = &b;
    printf("b: %lf\n", *(double*)vp); // 강제형변환후 *연산
    return 0;
}
```

a: 10  
b: 3.5



# main 함수로의 인자 전달

30

- 생략 -> 각자 공부해볼 것



- 함수 선언, 호출, 정의를 각각 설명하라
- 함수의 자료형은?
- 함수명의 자료형은?
- void 포인터의 용도는?
- void 포인터에 간접참조 연산을 적용할 때 주의할 점은?
- 강제형변환과 자동형변환을 설명하시오

- 인터넷에서 함수의 매개변수에 함수 포인터를 활용하는 예제를 찾아 실행해보고 코드를 설명하시오.
- 인터넷에서 함수의 매개변수에 void포인터를 활용하는 예제를 찾아 실행해보고 코드를 설명하시오.

- 예제 2번을 참고하여 함수포인터를 활용하여 다음처럼 실행되는 코드를 작성하시오.
- 공통부분은 하나의 함수에 작성하고 연산부분만 분리된 함수로 만들 것

연산을 선택하시오(1:덧셈,2:뺄셈,3:곱셈,4:나눗셈) : 1 <엔터>  
두개의 정수를 입력하시오 : 10 20 <엔터>  
결과값: 30



- 20장 도전문제 중에서 3문제만 골라서 푸시오.



# 과제제출방법

35

- 아래 주소의 작성방법대로 github에 제출할 것
  - ▣ <https://github.com/2sungryul/cprogramming/tree/main>
- 기한 : 이클래스 과제 게시판의 마감시간 참조
- 소스코드의 첫 부분은 아래처럼 제목,날짜,작성자(학번,이름)를 작성할 것

```
// *****  
//   제   목   : 정수 4개의 평균을 구하는 프로그램  
//   날   짜   : 2023년 9월10일  
//   작성자   : 15010101 홍길동  
// *****  
  
// 소스코드 작성
```